

PostgreSQL Internals Interview Guide

50 deep-dive Q&A covering storage, MVCC, WAL, VACUUM, query pipeline, and more. This guide targets Senior, Staff, and Principal Engineer interviews where interviewers expect precise byte-level knowledge, not just high-level descriptions.

Q1: Describe the physical layout of a PostgreSQL heap page. What are the major sections and their sizes?

Difficulty: Medium **Category:** Storage

Answer: Every PostgreSQL heap page is exactly 8,192 bytes (8 kB) by default, configurable only at compile time via `--with-blocksize`. The page begins with a `PageHeaderData` struct occupying 24 bytes, which stores the page LSN (8 bytes), checksum (2 bytes), flags (2 bytes), `pd_lower` (2 bytes, offset to free space start), `pd_upper` (2 bytes, offset to free space end), `pd_special` (2 bytes, offset to special space), and `pd_pagesize_version` (2 bytes). Immediately after the header is the item identifier (`ItemId`) array, where each entry is 4 bytes encoding an offset, length, and flags for one tuple. Tuples themselves are packed from the bottom of the page upward, so `pd_upper` shrinks as tuples are added. The region between `pd_lower` and `pd_upper` is free space. The special space at the end is used by index pages (e.g., B-tree stores opaque data there) but is zero-length for heap pages. This layout means that a fresh empty page has `pd_lower = 24` and `pd_upper = 8192`.

Follow-up Questions:

- How does PostgreSQL detect a torn page write using the checksum field?
- What happens to `pd_lower` and `pd_upper` when a tuple is deleted?

What Interviewer Looks For: Candidate should cite the 24-byte header size and name at least `pd_lower` / `pd_upper` without prompting. Bonus points for knowing each `ItemId` is 4 bytes and explaining the bidirectional fill pattern.

Common Mistakes: Candidates often say the page header is "a few bytes" without knowing 24 is the precise size. Confusing `pd_lower` (tracks item array growth) with `pd_upper` (tracks tuple data growth) is another frequent error.

Q2: What fields does a `HeapTupleHeaderData` contain and what is its minimum size?

Difficulty: Hard **Category:** Storage

Answer: `HeapTupleHeaderData` is the on-disk header prepended to every heap tuple. Its minimum size is 23 bytes, but alignment pads it to 24 bytes in practice. The fields are: `t_xmin` (4 bytes, XID of inserting transaction), `t_xmax` (4 bytes, XID of deleting/locking transaction, 0 if live), `t_cid` or `t_xvac` (4 bytes, command ID or VACUUM XID in a union), `t_ctid` (6 bytes, current TID — block number plus offset — pointing to the latest tuple version), `t_infomask2` (2 bytes, stores number-of-attributes in low bits and HOT/KEY-UPDATE flags in high bits), `t_infomask` (2 bytes, visibility hint bits and tuple-type flags), and `t_hoff` (1 byte, offset to user data, accounting for null bitmap). The null bitmap follows the fixed header if `HEAP_HASNULL` is set in `t_infomask`, adding `ceil(natts/8)` bytes. Understanding `t_infomask` bits like `HEAP_XMIN_COMMITTED`, `HEAP_XMAX_INVALID`, and `HEAP_HOT_UPDATED` is critical for understanding visibility and HOT chains.

Follow-up Questions:

- Why is `t_ctid` set to the tuple's own TID for a live non-updated tuple?
- What does the `HEAP_ONLY_TUPLE` flag in `t_infomask2` indicate?

What Interviewer Looks For: Precise knowledge of `t_xmin`, `t_xmax`, `t_ctid`, and both infomask fields. The interviewer expects the candidate to explain why `t_ctid` points forward in an update chain.

Common Mistakes: Forgetting that `t_cid` and `t_xvac` share a union and misreading the tuple size as 23 bytes without accounting for alignment padding to 24 bytes.

Q3: How does PostgreSQL use the ItemId array to manage free space within a page?

Difficulty: Medium **Category:** Storage

Answer: Each entry in the `ItemId` array (also called the line pointer array) is a 4-byte struct with three fields packed via bit fields: `lp_off` (15 bits, byte offset of tuple from page start), `lp_flags` (2 bits, `LP_UNUSED=0`, `LP_NORMAL=1`, `LP_REDIRECT=2`, `LP_DEAD=3`), and `lp_len` (15 bits, byte length of tuple). When a tuple is inserted, PostgreSQL appends a new `ItemId` at `pd_lower`, increments `pd_lower` by 4, and writes the tuple data at `pd_upper - tuple_size`, then decrements `pd_upper`. Free space is therefore the gap `pd_upper - pd_lower`. When a tuple is deleted, its `ItemId` is marked `LP_DEAD` but the slot is not immediately reclaimed — `VACUUM` later sets it to `LP_UNUSED` and the space can be reused. HOT updates change a dead `ItemId` to `LP_REDIRECT`, pointing to the new tuple's offset within the same page. This indirection layer allows index entries to remain valid across updates without index page modifications.

Follow-up Questions:

- How many tuples can fit on a single 8 kB page for a minimal-width row?
- What triggers a page compaction (defragmentation) of tuple space?

What Interviewer Looks For: The candidate must explain the bidirectional fill and the role of `LP_REDIRECT` in HOT chains. Understanding that `ItemId` slots survive tuple deletion is a key insight.

Common Mistakes: Assuming that deleting a tuple immediately reclaims page space; it does not until `VACUUM` runs. Also confusing `LP_DEAD` (logically dead, space not yet reclaimed) with `LP_UNUSED` (slot available for reuse).

Q4: What is the FILLFACTOR storage parameter and how does it affect page layout?

Difficulty: Medium **Category:** Storage

Answer: `FILLFACTOR` is a per-table or per-index storage parameter (1–100, default 100 for heap, 90 for B-tree indexes) that instructs PostgreSQL to leave a fraction of each page empty during initial inserts. For a heap table with `FILLFACTOR=70`, inserts fill only 70% of each page, reserving the remaining 30% for future updates. When a HOT update occurs — where all indexed columns are unchanged — PostgreSQL can place the new tuple version on the same page as the old version, enabling a HOT chain and avoiding index bloat. If there is no room on the same page for the new version, a non-HOT update occurs, requiring a new index entry pointing to the new page and causing index bloat. Tables with very frequent updates (e.g., counters, status columns) benefit enormously from lower fillfactor values such as 70–80. The downside is increased storage

consumption and potentially more pages to scan for sequential scans. Setting `FILLFACTOR` appropriately is one of the most impactful physical design decisions for write-heavy OLTP workloads.

Follow-up Questions:

- How does `FILLFACTOR` interact with autovacuum triggering thresholds?
- Can you change `FILLFACTOR` on an existing table without a full rewrite?

What Interviewer Looks For: The candidate should connect `FILLFACTOR` directly to HOT update eligibility. The interviewer expects an explanation of the space reservation mechanism and the trade-off between write performance and storage.

Common Mistakes: Stating that `FILLFACTOR` applies during all operations rather than only during initial inserts and `VACUUM` fills. Forgetting that changing `FILLFACTOR` with `ALTER TABLE` does not immediately reorganize existing pages; it only affects future inserts.

Q5: How does PostgreSQL calculate the physical location of a tuple given its TID?

Difficulty: Medium **Category:** Storage

Answer: A TID (Tuple Identifier, also called `ctid`) is a 6-byte struct consisting of a 4-byte block number (`BlockNumber`, 0-indexed) and a 2-byte offset number (`OffsetNumber`, 1-indexed). To read a tuple, PostgreSQL first calls `ReadBuffer` with the relation's file and the block number, which loads the 8 kB page into the shared buffer pool. It then dereferences `PageGetItemId(page, offset)` to retrieve the `ItemId` at position `offset` in the item array. The `ItemId`'s `lp_off` field gives the byte offset within the page where the `HeapTupleHeaderData` begins. PostgreSQL performs bounds checking to ensure `lp_off` is within the valid tuple area (between 24 and `pd_upper`). For `LP_REDIRECT` `ItemIds` (HOT chains), the `lp_off` is reinterpreted as the redirect target offset number, and the lookup follows the chain. This two-level indirection (block → `ItemId` → tuple data) is fundamental to PostgreSQL's storage architecture and allows in-place updates of `ItemIds` without touching index structures.

Follow-up Questions:

- What is the maximum table size given a 4-byte block number and 8 kB block size?
- How does `ctid` change when a row is moved by `CLUSTER` or `VACUUM FULL`?

What Interviewer Looks For: Precise decoding of block number plus offset number with correct sizes. The interviewer looks for mention of the shared buffer pool fetch and the two-level lookup via the `ItemId` array.

Common Mistakes: Confusing the offset number with a byte offset into the page rather than an index into the `ItemId` array. Also forgetting that offset numbers are 1-indexed, not 0-indexed.

Q6: Explain the concept of a "relation fork" in PostgreSQL storage. What forks exist?

Difficulty: Hard **Category:** Storage

Answer: PostgreSQL stores each relation (table, index, sequence, etc.) as a set of files called forks, each representing a different aspect of the relation's data. The main fork (fork number 0) contains the actual heap or index data pages. The free space map (FSM) fork (fork number 1) is a binary tree structure recording available space per page, enabling fast selection of pages with enough room for new tuples without scanning

every page. The visibility map (VM) fork (fork number 2) has one bit per page indicating whether all tuples on the page are visible to all transactions, enabling index-only scans and allowing VACUUM to skip pages that need no cleanup. The initialization fork (fork number 3) exists only for unlogged tables and contains a clean copy used to reinitialize the relation after a crash, since unlogged tables bypass WAL and lose data after a crash. Fork files are named as `<relfilenode>` , `<relfilenode>_fsm` , `<relfilenode>_vm` , and `<relfilenode>_init` in the relation's tablespace directory. Each fork is segmented into 1 GB files (suffix `<N>`) when it exceeds that size.

Follow-up Questions:

- How does the FSM binary tree encode free space per page in just a few bytes?
- When is the VM fork invalidated, and what must happen before it can be trusted again?

What Interviewer Looks For: Naming all four forks with their numbers and purposes. Emphasis on the VM fork's role in enabling index-only scans and skipping VACUUM work is especially valued.

Common Mistakes: Forgetting the initialization fork entirely or confusing it with the main fork. Describing the VM as a simple bitmap without explaining its role in both VACUUM optimization and index-only scans.

Q7: What is a TOAST table and when does PostgreSQL create one?

Difficulty: Medium **Category:** TOAST

Answer: TOAST stands for The Oversized-Attribute Storage Technique. PostgreSQL creates a companion TOAST table (named `pg_toast_<relid>`) for any user table that contains columns with a storage strategy other than PLAIN — specifically any `text` , `bytea` , `jsonb` , `hstore` , `arrays` , or other variable-length types that can exceed the inline threshold. The TOAST table is created automatically with the main table if at least one eligible column exists. It has three columns: `chunk_id` (OID linking back to the toasted value), `chunk_seq` (integer ordering the chunks), and `chunk_data` (bytea containing up to 2000 bytes of chunk data). The TOAST threshold is approximately 2,000 bytes (roughly one quarter of a page, computed as `TOAST_TUPLE_THRESHOLD = 2000`). When a row's total size would exceed this threshold, PostgreSQL attempts to compress eligible columns first (EXTENDED strategy), then moves them out-of-line to the TOAST table if still too large. Out-of-line values store an 18-byte "toast pointer" (`varattrib_1b_e` or `varattrib_4b` struct) in the main table's tuple referencing the TOAST table entries.

Follow-up Questions:

- How does PostgreSQL retrieve a toasted value during a query — is it always detoasted?
- What does the MAIN storage strategy do differently from EXTENDED?

What Interviewer Looks For: Knowing the 2,000-byte threshold and the 18-byte pointer structure. The interviewer also wants to hear about `chunk_id`/`chunk_seq`/`chunk_data` columns and how the lookup works.

Common Mistakes: Stating the TOAST threshold is 8 kB (the page size) rather than ~2 kB. Forgetting that TOAST attempts compression before out-of-line storage for the EXTENDED strategy.

Q8: How does PostgreSQL manage free space across pages using the Free Space Map?

Difficulty: Hard **Category:** Storage

Answer: The Free Space Map (FSM) fork stores a binary tree of byte values representing available free space on each heap page. The leaf nodes correspond directly to heap pages, each encoded as a single byte where

the value `v` represents `v * 32` bytes of free space (FSM_CATEGORIES = 256, so the maximum representable is $255 * 32 = 8,160$ bytes). Internal nodes store the maximum of their children, creating a max-heap structure that allows $O(\log N)$ search for a page with at least `needed` free bytes: start at the root and traverse to a leaf whose value is sufficient. When a tuple is inserted, `GetPageWithFreeSpace` walks the FSM tree using the required size, returning a block number. After a page is modified,

`RecordPageWithFreeSpace` updates the leaf and propagates the change upward through parent nodes. VACUUM updates the FSM after cleaning dead tuples by calling `FreeSpaceMapVacuum`. The FSM file grows in 8 kB pages just like the main heap, with each FSM page holding space data for up to ~4,000 heap pages. This design avoids scanning pages randomly looking for free space, which would be catastrophically slow for large tables.

Follow-up Questions:

- What happens to the FSM when VACUUM runs and finds newly freed space?
- Can the FSM become out of date? What are the consequences?

What Interviewer Looks For: The one-byte-per-page encoding (value $\times$ 32 bytes) and the max-heap binary tree structure. Understanding the $O(\log N)$ search property sets top candidates apart.

Common Mistakes: Describing the FSM as a simple bitmap or list rather than a tree structure. Forgetting the quantization effect of the byte encoding (free space is rounded down to the nearest 32-byte multiple).

Q9: Explain PostgreSQL's MVCC snapshot model. What data does a snapshot capture?

Difficulty: Medium **Category:** MVCC

Answer: A PostgreSQL MVCC snapshot captures the state of active transactions at the moment the snapshot is taken, allowing consistent reads without locking. A snapshot is a `SnapshotData` struct containing: `xmin` (the lowest XID that was active when the snapshot was taken — all XIDs below `xmin` are either committed or rolled back and have a definitive visibility), `xmax` (the first XID that was not yet assigned — any `XID >= xmax` is invisible as it started after the snapshot), and `xip` (an array of XIDs that were in-progress at snapshot time, i.e., between `xmin` and `xmax` but not yet committed). For `READ COMMITTED` isolation, a new snapshot is taken at the start of each statement. For `REPEATABLE READ` and `SERIALIZABLE`, the snapshot is taken once at the start of the first statement in the transaction and reused for all subsequent statements. The `cmin / cmax` command counters handle visibility within a single transaction for rows modified by prior commands in the same transaction. Snapshots are acquired cheaply via `GetTransactionSnapshot()` which reads `ShmemVariableCache->latestCompletedXid` and the `ProcArray`.

Follow-up Questions:

- How does the `xip` array affect memory usage for systems with many long-running transactions?
- What is the difference between a snapshot's `xmin` and a relation's `relfrozenxid`?

What Interviewer Looks For: Precise definition of the `xmin/xmax/xip` triple. The interviewer wants to hear how `READ COMMITTED` and `REPEATABLE READ` differ in snapshot acquisition frequency.

Common Mistakes: Confusing the snapshot's `xmin` with the tuple's `t_xmin`. These are different concepts: snapshot `xmin` is the oldest active transaction, while tuple `xmin` is the inserting transaction's XID.

Q10: Walk through the tuple visibility rules for MVCC. When is a tuple visible to a snapshot?

Difficulty: Hard **Category:** MVCC

Answer: PostgreSQL's visibility rules are implemented in `heap_hot_search_buffer` and `HeapTupleSatisfiesMVCC`. A tuple with `t_xmin = X` and `t_xmax = Y` is visible to snapshot `S` if all of the following hold: (1) `X` is committed — either the `HEAP_XMIN_COMMITTED` hint bit is set, or `TransactionIdDidCommit(X)` returns true (checked via `pg_xact`); (2) `X < S.xmax` — `X` started before the snapshot's upper bound; (3) `X` is not in `S.xip` — `X` was not in-progress when the snapshot was taken; (4) `Y` is either 0 (tuple not deleted), or `Y` is the current transaction (allows a transaction to see its own deletions), or `Y` is not committed (the deleter rolled back), or `Y >= S.xmax` (the deleter started after the snapshot), or `Y` is in `S.xip` (the deleter was in-progress). The hint bits `HEAP_XMIN_COMMITTED`, `HEAP_XMIN_INVALID`, `HEAP_XMAX_COMMITTED`, and `HEAP_XMAX_INVALID` cache the commit/abort status to avoid repeated `pg_xact` lookups. Setting a hint bit is a minor page modification that does not generate WAL for the hint bit itself (prior to `wal_level = logical`), though a full-page write may occur if the page is dirty.

Follow-up Questions:

- What happens if a transaction sets a hint bit on a page it does not own — is that safe?
- How does the command ID (`cmin/cmax`) affect visibility within a single transaction?

What Interviewer Looks For: All five conditions for visibility stated in order, especially the nuanced handling of `t_xmax`. The role of hint bits in caching commit status to avoid `pg_xact` lookups is a strong signal.

Common Mistakes: Omitting the `S.xip` check, which is the most subtle rule. Many candidates state "if `xmin` is committed and `xmax` is zero or not committed" but forget that a committed `xmin` in the `xip` array is invisible.

Q11: What is a HOT (Heap-Only Tuple) update and what conditions must be satisfied for it to occur?

Difficulty: Hard **Category:** MVCC

Answer: A HOT update is an optimization where an updated tuple is placed on the same heap page as the original tuple without creating a new index entry. Three conditions must all be satisfied: (1) the updated tuple fits on the same page as the original (requires free space, which `FILLFACTOR` reserves), (2) none of the columns covered by any index on the table are modified by the update, and (3) the new tuple version is placed on the same page. When these conditions hold, PostgreSQL marks the old tuple's `t_infomask2` with `HEAP_HOT_UPDATED` and the new tuple's `t_infomask2` with `HEAP_ONLY_TUPLE`. The old tuple's `t_ctid` points to the new tuple's TID. Existing index entries continue to point to the original tuple's TID. When an index scan finds the original TID, PostgreSQL follows the HOT chain (the sequence of `LP_REDIRECT` and `HEAP_ONLY_TUPLE` pointers) to locate the latest visible version. `VACUUM` can prune dead HOT chain members by converting their `ItemIds` to `LP_UNUSED` and redirecting the chain head's `ItemId` to `LP_REDIRECT` pointing directly to the surviving version, compressing the chain. HOT dramatically reduces index bloat and write amplification for updates that do not touch indexed columns.

Follow-up Questions:

- How does PostgreSQL find the root of a HOT chain when traversing from an index entry?
- What happens to a HOT chain when the surviving tuple is itself updated to a different page?

What Interviewer Looks For: All three conditions for HOT eligibility. The interviewer specifically looks for the `HEAP_HOT_UPDATED` and `HEAP_ONLY_TUPLE` flag distinction and explanation of how index entries remain valid.

Common Mistakes: Stating that HOT works for any update as long as there is free space on the page, forgetting the indexed-columns-unchanged requirement. Also forgetting that `FILLFACTOR` is the primary mechanism for ensuring free space availability.

Q12: How does PostgreSQL handle concurrent updates to the same row? Explain the "tuple locking" mechanism.

Difficulty: Hard **Category:** MVCC

Answer: When transaction T2 attempts to update a row that transaction T1 is currently updating, PostgreSQL uses `t_xmax` to detect the conflict. T1's update sets `t_xmax` to T1's XID on the old tuple version, effectively locking the row. T2 calls `heap_update`, which calls `HeapTupleSatisfiesUpdate` and finds `t_xmax = T1_XID` with `HEAP_XMAX_INVALID` not set. T2 then calls `XactLockTableWait(T1_XID)` to sleep until T1 commits or rolls back. If T1 commits, T2 re-examines the row: since the old version is now dead, T2 must update T1's new version (the row pointed to by `t_ctid`), re-evaluating any `WHERE` conditions in the `READ COMMITTED` level. If T1 rolls back, T2 can update the original row. For `SELECT FOR UPDATE/SHARE`, PostgreSQL uses a multi-transaction (MultiXact) mechanism when multiple transactions hold row locks simultaneously: `t_xmax` is set to a MultiXactId that encodes multiple lockers, and the MultiXact members are stored in `pg_multixact`. The infomask bits `HEAP_XMAX_IS_MULTI`, `HEAP_XMAX_EXCL_LOCK`, and `HEAP_XMAX_KEYSHR_LOCK` distinguish lock types.

Follow-up Questions:

- What is the difference between a key-share lock and an exclusive row lock in terms of infomask bits?
- How does PostgreSQL avoid deadlocks in the row-locking scenario?

What Interviewer Looks For: Understanding that `t_xmax` serves dual duty as both a delete marker and a lock marker. The MultiXact mechanism for multiple concurrent row locks is a strong differentiator.

Common Mistakes: Describing row locking as a separate lock table entry rather than as an in-place `t_xmax` modification. Forgetting the MultiXact mechanism for concurrent readers holding `FOR SHARE` locks.

Q13: What is XID wraparound and why is it dangerous?

Difficulty: Hard **Category:** MVCC

Answer: PostgreSQL transaction IDs (XIDs) are 32-bit unsigned integers, providing approximately 4 billion unique values. The XID space is divided into a circular sequence using modular arithmetic: $XID \ A$ is considered "in the past" relative to B if $(B - A) \bmod 2^{32} < 2^{31}$. This means every XID has exactly $2^{31} \approx 2.1$ billion XIDs "in the past" and 2.1 billion "in the future." XID wraparound becomes dangerous when a table contains tuples whose `t_xmin` is so old that, after wraparound, their XID appears to be "in the future" rather than "in the past" — making all tuples on that table invisible to all new transactions, effectively causing data loss. PostgreSQL prevents this by requiring `VACUUM FREEZE` to replace old XIDs with the special FrozenTransactionId (XID 2), which is always considered older than any normal XID. The parameter `autovacuum_freeze_max_age` (default 200,000,000 XIDs) controls how aggressively autovacuum freezes tuples. PostgreSQL will forcibly shut down (with an error "database is not accepting commands to

avoid wraparound data loss") when any XID is within 1,000,000 transactions of wraparound, detectable in `pg_database.datfrozenxid` and `pg_class.relFrozenxid` .

Follow-up Questions:

- How do you check the current XID age of a database and identify tables at wraparound risk?
- What is the difference between `vacuum_freeze_min_age` and `autovacuum_freeze_max_age` ?

What Interviewer Looks For: The circular modular arithmetic explanation and the consequence of tuples becoming invisible. Knowing the 200 million XID default and the forced-shutdown behavior demonstrates operational depth.

Common Mistakes: Saying wraparound "corrupts data" without explaining the visibility mechanism by which tuples become invisible. Also forgetting that FrozenXID (XID 2) is the special value used for freezing, not just "marking as old."

Q14: Explain dead tuple accumulation. How does it affect query performance and what metrics expose it?

Difficulty: Medium **Category:** MVCC

Answer: When a row is updated or deleted in PostgreSQL, the old tuple version remains on the heap page because other transactions may still need to read it (MVCC visibility). These old versions are called dead tuples once no snapshot can see them anymore. Dead tuples waste space on heap pages, reducing the effective data density per page and forcing sequential scans to read more pages than necessary for the actual live row count. They also bloat indexes, which hold pointers to dead tuple TIDs that must be checked during index scans. The `pg_stat_user_tables` view exposes `n_dead_tup` (estimated dead tuple count from autovacuum statistics) and `n_live_tup` . The ratio `n_dead_tup / (n_live_tup + n_dead_tup)` measures bloat percentage. `pg_class.relpages` and `pg_class.reltuples` (updated by VACUUM/ANALYZE) let you estimate page-level bloat. Tools like `pgstattuple` extension provide exact dead tuple byte counts and counts via `pgstattuple('tablename')` . Dead tuples trigger autovacuum when `n_dead_tup > autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples` . High dead tuple counts on hot tables indicate insufficient autovacuum aggressiveness.

Follow-up Questions:

- How does a long-running transaction prevent dead tuple removal even after VACUUM runs?
- What is the difference between `n_dead_tup` in `pg_stat_user_tables` and the output of `pgstattuple` ?

What Interviewer Looks For: Connection between dead tuples, page bloat, and sequential scan performance. Knowing the autovacuum trigger formula with `autovacuum_vacuum_threshold` and `autovacuum_vacuum_scale_factor` is a strong indicator.

Common Mistakes: Stating that VACUUM removes dead tuples immediately without noting the blocking condition: dead tuples cannot be removed if any active snapshot still references them. Failing to mention the performance impact of index bloat alongside heap bloat.

Q15: How does PostgreSQL implement READ COMMITTED vs REPEATABLE READ at the storage level?

Difficulty: Hard **Category:** MVCC

Answer: At the storage level, the difference between READ COMMITTED and REPEATABLE READ is purely about when a new snapshot is acquired. In READ COMMITTED, `GetTransactionSnapshot()` is called at the beginning of each individual SQL statement, producing a fresh `SnapshotData` with the current `xmax` and `xip` array. This means each statement sees all commits that occurred before that statement started, which can lead to non-repeatable reads (the same row returning different values in successive statements within one transaction). In REPEATABLE READ, `GetTransactionSnapshot()` is called only once per transaction (lazily, on the first statement that needs a snapshot) and the resulting `SnapshotData` is stored in `CurrentSnapshotData` and reused for all subsequent statements. Since `xmax` and `xip` are fixed at that point, any rows committed after the first statement's snapshot was taken are invisible. SERIALIZABLE adds conflict detection via the SSI (Serializable Snapshot Isolation) mechanism, which tracks read/write dependencies in `SERIALIZABLEXACT` structs and aborts transactions when dangerous cycles are detected. No additional storage-level locking is used for SERIALIZABLE beyond the dependency tracking.

Follow-up Questions:

- Can a REPEATABLE READ transaction see its own uncommitted changes? How?
- What is a "dangerous structure" in SSI and how does PostgreSQL detect it?

What Interviewer Looks For: The exact point of snapshot acquisition (per-statement vs per-transaction) and the concrete implication for anomaly prevention. Mentioning SSI for SERIALIZABLE goes beyond what most candidates know.

Common Mistakes: Describing the difference in terms of "locking" rather than snapshot timing. REPEATABLE READ in PostgreSQL uses snapshot visibility, not read locks, unlike some other databases.

Q16: What is the `pg_xact` directory and how does PostgreSQL use it for commit status?

Difficulty: Hard **Category:** MVCC

Answer: `pg_xact` (previously named `pg_clog` before PostgreSQL 10) is a directory containing SLRU (Simple Least Recently Used) cache files that record the commit status of every transaction. Each transaction requires 2 bits of storage: the status can be `IN_PROGRESS` (00), `COMMITTED` (01), `ABORTED` (10), or `SUB_COMMITTED` (11, used for subtransaction tracking). The files are named as 4-digit hexadecimal segment numbers (e.g., `0000`, `0001`) and each file is 256 kB, storing status for $256 \text{ kB} * 8 / 2 = 1,048,576$ transactions. When PostgreSQL needs to determine whether `XID X` is committed (e.g., during visibility checking if hint bits are absent), it calls `TransactionIdGetStatus(X)`, which reads the appropriate byte from the SLRU cache. The SLRU cache holds 32 buffers of 8 kB each (configurable via `transaction_status_cache_size` in newer versions). Old `pg_xact` segments are removed by `VACUUM` once all transactions in that segment are known to be either committed or aborted and all `datfrozenxid` values have advanced past that range. This two-bit scheme makes commit status lookups extremely efficient — a single byte read covers four transaction statuses.

Follow-up Questions:

- What is a subtransaction and how does it affect `pg_xact` storage?
- How does PostgreSQL handle `pg_xact` reads during crash recovery?

What Interviewer Looks For: The 2-bit encoding and the SLRU architecture. Knowing the file naming convention and segment size shows deep familiarity with PostgreSQL source code.

Common Mistakes: Confusing `pg_xact` (commit status) with `pg_xid` (which does not exist) or with the WAL. Also stating that `pg_xact` uses a hash table rather than a sequential bit array organized by `XID`.

Q17: Describe the four TOAST storage strategies: PLAIN, EXTENDED, EXTERNAL, and MAIN.

Difficulty: Medium **Category:** TOAST

Answer: PostgreSQL's TOAST system provides four storage strategies that control how oversized column values are handled. `PLAIN` stores values inline on the heap page without compression or out-of-line storage; this strategy is mandatory for fixed-length types (integer, float, etc.) and for columns where the DBA explicitly disables TOAST. `EXTENDED` (the default for most variable-length types like `text`, `jsonb`, `bytea`) first tries to compress the value using LZ4 or pglz; if the compressed value still does not fit inline, it is stored out-of-line in the TOAST table. `EXTERNAL` stores the value out-of-line in the TOAST table without compression, which increases storage size but allows PostgreSQL to retrieve only a substring of a large value without decompressing the whole thing — useful for `bytea` columns where partial access via `substring()` is common. `MAIN` tries compression first like `EXTENDED`, but prefers to keep the value inline even if it is oversized; out-of-line storage is used only as a last resort, making it suitable for columns that are almost always small but occasionally large. Strategies can be changed with `ALTER TABLE t ALTER COLUMN c SET STORAGE strategy`.

Follow-up Questions:

- When would you choose `EXTERNAL` over `EXTENDED` for a large text column?
- How does pglz compression compare to LZ4 in PostgreSQL 14+?

What Interviewer Looks For: Distinguishing `EXTERNAL` (no compression, partial detoast possible) from `EXTENDED` (compression first). The interviewer looks for understanding of why `EXTERNAL` enables efficient substring operations.

Common Mistakes: Conflating `EXTENDED` and `MAIN` — both compress first but differ in their preference for inline vs out-of-line storage when the value is borderline. Also forgetting that `PLAIN` cannot be applied to variable-length types that can exceed the page size.

Q18: How is a TOAST pointer structured and what information does it contain?

Difficulty: Hard **Category:** TOAST

Answer: When a value is stored out-of-line in a TOAST table, a TOAST pointer (also called an indirect or external varlena) replaces the actual value in the main heap tuple. The standard TOAST pointer is 18 bytes stored as a `varattrib_4b` struct with a special tag byte. It contains: `va_rawsize` (4 bytes, uncompressed data size), `va_extsize` (4 bytes, size of the external data in the TOAST table), `va_valueid` (4 bytes, OID of the TOAST entry — the `chunk_id`), and `va_toastrelid` (4 bytes, OID of the TOAST table relation, from `pg_class`). Additionally, a compression indicator bit in the tag byte signals whether the external data is pglz/LZ4 compressed. When a query retrieves a column with a TOAST pointer, `heap_tuple_untoast_attr` is called, which executes an internal query against the TOAST table: `SELECT chunk_data FROM pg_toast_<oid> WHERE chunk_id = va_valueid ORDER BY chunk_seq` to reassemble the chunks. An in-memory TOAST pointer (`va_datum_kind = TOAST_MEMORY`) can also exist for live in-memory data, but is never written to disk.

Follow-up Questions:

- How does detoasting behave during an index-only scan that never touches the heap?

- What happens to TOAST pointers when a table is pg_dump'd and restored?

What Interviewer Looks For: The 18-byte size and the four fields (rawsize, extsize, valueid, toastrelid). Understanding that toastrelid makes the pointer self-contained (no catalog lookup needed to find the TOAST table) is a strong indicator.

Common Mistakes: Stating that TOAST pointers contain the actual data or that they are just OIDs. Forgetting that the TOAST table OID is embedded in the pointer itself, allowing direct access without going through the system catalog.

Q19: How does PostgreSQL chunk data in the TOAST table? What is the maximum chunk size?

Difficulty: Medium **Category:** TOAST

Answer: TOAST table rows contain chunks of the original oversized value stored in the `chunk_data` bytea column. The maximum chunk size is `TOAST_MAX_CHUNK_SIZE = 2000` bytes, which is chosen so that each TOAST row fits comfortably within a single heap page (the 2000-byte chunk plus the tuple header and other column values stays well under 8 kB). Large values are split into N chunks numbered from 0 to N-1 in `chunk_seq`. When PostgreSQL stores a toasted value, it loops through the data in 2000-byte increments, inserting one TOAST row per chunk. Reassembly requires fetching all chunks ordered by `chunk_seq` and concatenating `chunk_data`. The TOAST table has a unique index on `(chunk_id, chunk_seq)` which ensures chunk ordering is enforced at the storage level and allows efficient range scans during detoasting. Deleting a value from the main table does not immediately remove TOAST rows; VACUUM on the main table triggers `toast_delete_datum` which deletes corresponding TOAST table entries. TOAST tables also have their own autovacuum activity tracked in `pg_stat_user_tables` and can independently develop bloat.

Follow-up Questions:

- Can TOAST tables themselves become bloated, and how do you identify TOAST table bloat?
- How does PostgreSQL handle compression of small chunks vs. the entire value before chunking?

What Interviewer Looks For: The exact 2000-byte chunk size and understanding that the TOAST unique index on `(chunk_id, chunk_seq)` enforces correct ordering. The connection between main table VACUUM and TOAST row deletion is important.

Common Mistakes: Stating the chunk size as 8 kB or 4 kB rather than 2000 bytes. Assuming TOAST cleanup is automatic without VACUUM on the main table, leading to TOAST bloat not being associated with the parent table's maintenance.

Q20: When does PostgreSQL decide to compress a TOAST value versus store it uncompressed out-of-line?

Difficulty: Hard **Category:** TOAST

Answer: The decision to compress or store uncompressed out-of-line depends on the column's storage strategy and the result of a compression attempt. For `EXTENDED` strategy (the most common), when a tuple is too large (total tuple size exceeds `TOAST_TUPLE_THRESHOLD ≈ 2000` bytes), PostgreSQL iterates through all `EXTENDED` and `MAIN` columns in a priority order (largest first) and attempts compression using the configured `default_toast_compression` algorithm (pglz prior to PG14, then LZ4 became available). If the compressed size is less than the original size (specifically if it is below `TOAST_COMPRESSION_THRESHOLD`), the compressed version replaces the column inline — no TOAST table needed. Only if the compressed value still

makes the tuple too large does PostgreSQL move it to the TOAST table. For `EXTERNAL` strategy, compression is skipped entirely and the value goes directly to the TOAST table uncompressed. The compression format is recorded in the varlena header: a `VARATT_IS_COMPRESSED` tag bit distinguishes compressed inline data from plain data. In PostgreSQL 14+, the compressor can be specified per-column: `ALTER TABLE t ALTER COLUMN c SET COMPRESSION lz4`.

Follow-up Questions:

- What query can you run to determine which compression method is in use for a column's stored values?
- How does LZ4 compression affect detoasting performance compared to pglz?

What Interviewer Looks For: The two-step process: compress first (inline), then move out-of-line. Knowing that PG14 added per-column compression method selection and LZ4 support differentiates senior candidates.

Common Mistakes: Describing compression and out-of-line storage as mutually exclusive rather than sequential steps. Forgetting that `EXTERNAL` bypasses compression entirely, making it suboptimal for storage but optimal for partial-access patterns.

Q21: How does VACUUM handle TOAST tables? Are they vacuumed separately or together?

Difficulty: Medium **Category:** TOAST

Answer: TOAST tables are vacuumed as a side effect of vacuuming the parent table, not independently via autovacuum scheduling (though they do appear in `pg_class` and technically can be vacuumed directly). When `VACUUM` runs on a main table, after processing the heap pages, it calls `toast_vacuum_scan_entries` which scans the dead tuple TIDs collected during the heap scan and deletes corresponding TOAST chunks for any deleted toasted values. This ensures TOAST orphans — chunks whose parent main-table tuple no longer exists — are cleaned up in the same `VACUUM` pass. However, autovacuum does also schedule `VACUUM` on TOAST tables independently based on their own `pg_stat_user_tables` stats (dead tuple count, last vacuum time). This can occasionally cause confusion: a TOAST table appears in autovacuum logs with its `pg_toast` name. If the main table's autovacuum is too infrequent, TOAST bloat can accumulate independently. The `pg_toast` schema is visible via `\dt pg_toast.*` in psql and TOAST table sizes are queryable via `pg_total_relation_size(oid)` which includes TOAST by default, unlike `pg_relation_size`.

Follow-up Questions:

- How do you identify which main table a `pg_toast` table belongs to?
- Can you run `CLUSTER` on a TOAST table to defragment it?

What Interviewer Looks For: Understanding that TOAST cleanup is triggered by main table `VACUUM` but that TOAST tables also have independent autovacuum scheduling. Knowing that `pg_total_relation_size` includes TOAST size is a practical operational detail.

Common Mistakes: Believing that TOAST tables are never independently vacuumed and only cleaned up through the parent. Also confusing `pg_relation_size` (excludes TOAST) with `pg_total_relation_size` (includes TOAST and indexes).

Q22: Describe a scenario where TOAST causes unexpected query slowness and how to diagnose it.

Difficulty: Hard **Category:** TOAST

Answer: A common TOAST performance trap occurs when a query `SELECTs` a toasted column that it does not actually need in the final output. Consider a table with a `jsonb metadata` column (toasted) and a query `SELECT id, name FROM t WHERE status = 'active'` that uses a covering index on `(status, id, name)` — this should be an index-only scan, but if the query plan falls back to a heap fetch and the planner estimates many rows, it may detoast the `metadata` column unnecessarily when fetching the row (though PostgreSQL does defer detoasting to the point of actually accessing the attribute). A more concrete problem: if a query does `SELECT *` or explicitly references the toasted column on millions of rows, each detoast requires multiple random I/Os to the TOAST table (one per chunk, plus index lookups on `pg_toast`). Diagnosis steps: (1) `EXPLAIN (ANALYZE, BUFFERS)` to count buffer hits/misses — excessive misses against `pg_toast_<oid>` pages show up in the TOAST table's buffer stats; (2) `pg_statio_user_tables` for the TOAST table's `heap_blks_read / heap_blks_hit`; (3) `pg_stat_user_tables` for the TOAST table showing high `seq_scan` counts. Mitigation: use `SELECT` only required columns, add storage `EXTERNAL` for partial-access columns, or use generated columns to pre-extract commonly queried subfields.

Follow-up Questions:

- How does PostgreSQL handle detoasting during a sort operation that spills to disk?
- Can you create an index on a toasted column? What are the limitations?

What Interviewer Looks For: Connecting the symptom (unexpected I/O) to the cause (detoasting via TOAST table reads). Knowing to look at `pg_statio_user_tables` for the toast table itself is a practical diagnostic detail that separates experienced engineers.

Common Mistakes: Assuming detoasting only happens when explicitly needed — forgetting that `SELECT *` triggers detoasting of all columns including unnecessary toasted ones. Overlooking that TOAST reads appear in buffer statistics as separate from the main table's stats.

Q23: What is a WAL LSN and how is it structured?

Difficulty: Medium **Category:** WAL

Answer: LSN stands for Log Sequence Number and is a 64-bit (8-byte) monotonically increasing integer representing a byte position in the WAL stream. It is split into a high 32-bit segment number and a low 32-bit offset within the segment. In PostgreSQL's display format (e.g., `2/3AF001C0`), the part before the slash is the segment number in hexadecimal and the part after is the byte offset within the segment. The default WAL segment size is 16 MB (`wal_segment_size`), configurable at initdb time. WAL files are named by their starting LSN, e.g., `000000010000000200000003`. An LSN uniquely identifies a position in the WAL and is used for: crash recovery (restart from the last checkpoint's redo LSN), replication (primary sends WAL from the replica's `flush_lsn` forward), and point-in-time recovery (target LSN or timestamp). Key LSN functions include `pg_current_wal_lsn()` (current write position), `pg_current_wal_flush_lsn()` (durably flushed), and `pg_walfile_name_offset(lsn)` (converts LSN to filename+offset). The difference `pg_current_wal_lsn() - sent_lsn` from `pg_stat_replication` measures replication lag in bytes.

Follow-up Questions:

- How do you convert a WAL filename back to the starting LSN of that segment?

- What is the significance of the `redo_lsn` stored in checkpoint records?

What Interviewer Looks For: The 64-bit structure split into segment+offset, the display format explanation, and practical usage in replication monitoring. Knowing `pg_current_wal_lsn()` vs `pg_current_wal_flush_lsn()` is a strong differentiator.

Common Mistakes: Stating LSN is 32-bit rather than 64-bit. Confusing the WAL filename (which encodes timeline + segment) with the LSN value itself.

Q24: Describe the structure of a WAL record. What are its major components?

Difficulty: Hard **Category:** WAL

Answer: Each WAL record begins with an `XLogRecord` header (24 bytes in PostgreSQL 9.3+, reduced from the earlier format) containing: `xl_tot_len` (4 bytes, total record length including header), `xl_xid` (4 bytes, XID of the generating transaction, or 0 for non-transactional records), `xl_prev` (8 bytes, LSN of the previous record — used for backward scan during recovery), `xl_info` (1 byte, record-type flags, resource manager specific), `xl_rmid` (1 byte, resource manager ID — e.g., `RM_HEAP_ID=0`, `RM_BTREE_ID=2`, `RM_XLOG_ID=0`), and a CRC32 checksum (4 bytes). After the header comes a sequence of `XLogRecordBlockHeader` entries (if the record references specific relation pages), each containing the relation's filenode, fork, block number, and whether a full-page image (FPI) is included. Full-page images are 8 kB (or compressed) copies of the page taken on the first modification after a checkpoint, ensuring idempotent replay. Finally, the record body contains the actual redo data (e.g., the new tuple data, WAL action code). Resource manager IDs map to modules like Heap, Index, Sequence, XLOG (checkpoint/control records), etc.

Follow-up Questions:

- Why does each WAL record include the previous record's LSN (`xl_prev`)?
- How does `xl_rmid` enable parallel WAL replay in PostgreSQL?

What Interviewer Looks For: The 24-byte header structure naming `xl_tot_len`, `xl_xid`, `xl_prev`, `xl_rmid`, and the CRC. Understanding the role of full-page images (FPI) after checkpoints is critical for crash safety.

Common Mistakes: Omitting the CRC field, which is critical for detecting corrupted WAL records during recovery. Also forgetting that `xl_prev` is used for backward scanning (e.g., during a transaction's rollback replay).

Q25: What are full-page writes (FPW) and why are they necessary?

Difficulty: Medium **Category:** WAL

Answer: Full-page writes (FPW) are a WAL safety mechanism where PostgreSQL records a complete 8 kB copy of a page in the WAL on the first modification of that page after each checkpoint. They are necessary because most storage hardware (HDDs, SSDs, storage arrays) writes data in units smaller than 8 kB (commonly 512-byte sectors or 4 kB physical sectors), creating the possibility of a "torn page write" during a crash: part of the new version and part of the old version of the page appear on disk. Without FPW, a partial WAL replay might apply a WAL delta record (which encodes the change as an offset+data diff) on top of a corrupted page, producing incorrect results. With FPW, the first WAL record for a page after a checkpoint contains the full page image, ensuring that recovery can start with a known-good page regardless of what the on-disk page looks like. FPW is enabled by `full_page_writes = on` (default). It increases WAL volume substantially — especially on write-intensive workloads — because a single UPDATE to a 100-byte row generates 8 kB+ of WAL.

for the first modification after a checkpoint. FPW data can be compressed (`wal_compression = on`) to reduce the overhead while maintaining safety.

Follow-up Questions:

- How does `wal_init_zero = off` interact with full-page writes?
- What is the performance impact of lowering `checkpoint_completion_target` ?

What Interviewer Looks For: The "torn page" motivation and the "first modification after checkpoint" trigger rule. Understanding that FPW increases WAL volume and that `wal_compression` mitigates this is a practical engineering insight.

Common Mistakes: Thinking FPW records every write rather than only the first write per page per checkpoint interval. Forgetting that `wal_compression` can significantly reduce the storage cost of FPW.

Q26: Explain the different `wal_level` settings and what each enables.

Difficulty: Medium **Category:** WAL

Answer: `wal_level` controls the amount of information written to WAL, with higher levels enabling more replication and recovery features at the cost of increased WAL volume. `minimal` writes only what is necessary for crash recovery — certain bulk operations like `COPY` and `CREATE TABLE AS` skip WAL entirely, making recovery possible but preventing streaming replication or archival. `replica` (the default since PostgreSQL 9.6, previously called `archive` and `hot_standby`) writes enough WAL to support streaming replication and WAL archiving; standby servers can connect and maintain a hot standby for read queries. `logical` extends `replica` with information needed by logical decoding — row-level change data including the old values of updated/deleted rows (using `REPLICA IDENTITY`). Logical replication, pglogical, and CDC tools like Debezium require `logical` . Setting `wal_level = logical` also writes column information into the WAL, increasing its size further. The level can be changed with a server restart; you cannot lower it while replicas are connected without first stopping them. Most production systems use `replica` unless logical replication is required, and `minimal` is only suitable for single-server batch ETL workloads.

Follow-up Questions:

- What is the minimum `wal_level` required for `pg_basebackup` to work correctly?
- How does `REPLICA IDENTITY FULL` change the WAL output for `UPDATE` statements?

What Interviewer Looks For: The three levels (minimal/replica/logical) with clear use-case mapping. Understanding that logical replication requires `logical` level and what additional information that writes is a differentiator.

Common Mistakes: Stating that `replica` level is necessary for PITR (actually, `replica` is the minimum, and `minimal` prevents archiving). Confusing `wal_level = hot_standby` (old name removed in PG10) with the current naming.

Q27: What is WAL archiving and how does `archive_command` work?

Difficulty: Medium **Category:** WAL

Answer: WAL archiving is the process of copying completed WAL segment files to a remote location (NFS, S3, GCS, etc.) for point-in-time recovery or as a fallback for streaming replication gaps. It is enabled by setting `archive_mode = on` and specifying `archive_command` , a shell command that PostgreSQL executes for each completed WAL segment. The command receives `%p` (full path to the WAL file) and `%f` (filename only)

substitutions, e.g., `archive_command = 'aws s3 cp %p s3://my-bucket/wal/%f'`. The archiver process runs as a background daemon and calls the command when a segment is complete (filled with 16 MB of WAL) or when `pg_switch_wal()` is called. The command MUST return exit code 0 to indicate success; non-zero means failure, and PostgreSQL will retry indefinitely while keeping the segment in `pg_wal/archive_status/` with a `.ready` suffix. Failed segments accumulate a `.failed` suffix. `pg_stat_archiver` tracks `archived_count`, `failed_count`, `last_archived_wal`, and `last_failed_wal`. WAL segments that are successfully archived get a `.done` status file. The critical safety rule: if `archive_command` succeeds (exit 0) but the file was not actually stored, data loss can occur.

Follow-up Questions:

- What is `archive_cleanup_command` and how is it different from `archive_command`?
- How does pgBackRest improve upon the basic `archive_command` mechanism?

What Interviewer Looks For: The `%p` / `%f` substitution syntax and the exit-code-0 success contract. Understanding the `.ready` / `.done` / `.failed` status file mechanism in `pg_wal/archive_status/` demonstrates operational familiarity.

Common Mistakes: Assuming non-zero exit causes data loss immediately — it only means the segment is retried, not lost (as long as the file stays in `pg_wal`). Forgetting that `archive_mode` must be combined with `wal_level >= replica` for archiving to work.

Q28: Describe the WAL writer process and its role in write performance.

Difficulty: Hard **Category:** WAL

Answer: The WAL writer is a PostgreSQL background process (started by the postmaster) responsible for flushing WAL buffer contents to WAL segment files on disk at regular intervals (`wal_writer_delay`, default 200ms, and `wal_writer_flush_after`, default 1MB). Without the WAL writer, flushing would only happen when a transaction commits (synchronous commit) or when a backend flushes explicitly. For asynchronous-commit workloads (`synchronous_commit = off`), the WAL writer ensures that WAL is periodically flushed without each transaction paying a sync cost, bounding the maximum data loss window to $2 * wal_writer_delay \approx 400ms$. For synchronous commits, individual backends still call `XLogFlush` on commit to ensure durability, but the WAL writer reduces redundant flushes by keeping WAL mostly current. The WAL writer works by acquiring the `WALWriteLock`, advancing `LogwrtResult.Write` (written to OS buffer), and then calling `pg_flush_data` followed by `issue_xlog_fsync` to advance `LogwrtResult.Flush` (durable). It cooperates with the WAL sender processes for replication. The `pg_stat_bgwriter` view (older) and `pg_stat_wal` (PostgreSQL 14+) expose `wal_write`, `wal_sync`, and `wal_write_time` metrics.

Follow-up Questions:

- How does `wal_sync_method` affect the system call used by the WAL writer during flush?
- What is the impact of setting `synchronous_commit = off` on WAL writer behavior?

What Interviewer Looks For: The WAL writer's role in reducing per-transaction fsync overhead and its relationship to `synchronous_commit`. Knowing `wal_writer_delay` and `wal_writer_flush_after` as the tuning knobs is practical knowledge.

Common Mistakes: Confusing the WAL writer with the WAL archiver (different processes with different jobs). Thinking the WAL writer is only needed for async commits — it also reduces fsync storms for synchronous commits.

Q29: What is the difference between WAL write and WAL flush?

Difficulty: Medium **Category:** WAL

Answer: WAL write and WAL flush are two distinct durability stages in PostgreSQL's write path. A WAL write copies data from PostgreSQL's in-memory WAL buffers (`wal_buffers` , default 4 MB or 1/32 of `shared_buffers`) to the OS page cache via `write()` system calls. After a write, the data resides in kernel memory but is not guaranteed to survive a power failure or OS crash — only an application crash would be survived. A WAL flush (also called sync) calls `fsync()` , `fdatasync()` , `open_datasync()` , or `O_DSYNC` (depending on `wal_sync_method`) on the WAL file, forcing the OS to flush its page cache to durable storage. For synchronous commits, a flush must complete before the `COMMIT` returns to the client. The LSN tracking distinguishes the two: `LogwrResult.Write` tracks the highest byte written to the OS, while `LogwrResult.Flush` tracks the highest byte durably synced. Replication uses both: the `pg_stat_replication` view shows `write_lsn` (replica has written to its WAL file), `flush_lsn` (replica has flushed to disk), and `replay_lsn` (replica has applied to its database). These three LSNs allow fine-grained control over `synchronous_standby_names` durability guarantees.

Follow-up Questions:

- Which `wal_sync_method` is fastest on Linux with modern SSDs?
- How does `synchronous_standby_names` interact with write vs. flush confirmation?

What Interviewer Looks For: The write = OS buffer, flush = durable distinction. Knowing all three LSN states (write/flush/replay) in `pg_stat_replication` and their meaning for synchronous replication shows depth.

Common Mistakes: Using "write" and "flush" interchangeably. Forgetting that replication confirmation can be requested at the write or flush level, not just the apply level.

Q30: How does PostgreSQL implement crash recovery using WAL?

Difficulty: Hard **Category:** WAL

Answer: PostgreSQL crash recovery is an ARIES-inspired redo-only protocol (unlike ARIES which also has an undo phase). On startup after a crash, the postmaster checks `pg_control` for the latest checkpoint record's redo LSN . Recovery begins by reading WAL from the redo LSN forward, applying every redo record in order to bring data files to a consistent state. Each WAL record's resource manager (`xl_rmid`) dispatches the record to the appropriate redo function (e.g., `heap_redo` , `btree_redo`). Full-page images embedded in WAL are applied first (replacing the potentially corrupted on-disk page with the known-good copy), then subsequent delta records are applied on top. Recovery ends when the end of WAL is reached (or a recovery target LSN/time/XID for PITR). Transactions that were in-progress at crash time are rolled back using `pg_xact` abort records or by marking them as aborted in `pg_xact` . PostgreSQL does NOT need an undo log for crash recovery because MVCC old versions remain on disk — in-progress transactions simply become "aborted" and their tuples are invisible to new transactions via the normal visibility rules. The `pg_control` file is updated with a new checkpoint once recovery completes.

Follow-up Questions:

- Why doesn't PostgreSQL need an undo log for crash recovery, unlike Oracle?
- How does recovery differ between a hard crash and a controlled shutdown?

What Interviewer Looks For: The redo-only protocol and the MVCC-based implicit undo mechanism. Knowing recovery starts from the checkpoint redo LSN and proceeds forward through all WAL is fundamental.

Common Mistakes: Stating that PostgreSQL performs undo like Oracle using a rollback segment. PostgreSQL's MVCC design means old tuple versions handle the "undo" implicitly by remaining invisible.

Q31: What is `synchronous_commit` and what are its trade-offs?

Difficulty: Medium **Category:** WAL

Answer: `synchronous_commit` controls how long `COMMIT` waits for WAL to be durably written before returning to the client. It has five settings: `on` (default — waits for WAL to be flushed to local disk, guaranteeing no data loss on crash), `remote_write` (waits for WAL to be written to the OS buffer on at least one synchronous standby, not flushed — protects against primary crash but not standby OS crash), `remote_apply` (waits for the standby to apply the WAL, ensuring replicas are up to date before the primary acknowledges), `local` (same as `on` but ignores standby confirmation — useful when replicas exist but you only care about local durability), and `off` (returns immediately without waiting for WAL flush — trades up to $2 * \text{wal_writer_delay}$ data loss risk for lower commit latency). `synchronous_commit = off` does NOT corrupt data: if PostgreSQL crashes before flushing, only the most recent ~400ms of committed transactions are lost, but the database remains internally consistent. It can be set per-session or per-transaction, allowing high-throughput bulk loads to use `off` while critical financial transactions use `on`.

Follow-up Questions:

- Can you set `synchronous_commit` per transaction? Give an example use case.
- How does `synchronous_standby_names` interact with `synchronous_commit = remote_write`?

What Interviewer Looks For: All five settings enumerated with their exact durability guarantees.

Emphasizing that `synchronous_commit = off` risks data loss but NOT corruption is a key nuance many candidates miss.

Common Mistakes: Conflating `synchronous_commit = off` with data corruption risk. Also forgetting the `remote_apply` and `remote_write` intermediate settings, which are valuable for replication-aware durability guarantees.

Q32: How does WAL compression work and when should you enable it?

Difficulty: Medium **Category:** WAL

Answer: `wal_compression` (PostgreSQL 9.5+) compresses full-page images (FPIs) embedded in WAL records using pglz, or LZ4/Zstandard (PostgreSQL 15+). FPIs are the dominant contributor to WAL volume on write-intensive systems — a checkpoint followed by many small updates can result in WAL being dominated by 8 kB page copies rather than the small deltas. Enabling `wal_compression` compresses each FPI before embedding it in the WAL record, reducing WAL volume by 50–80% in typical OLTP workloads. The decompression happens during WAL replay (recovery or standby), adding CPU cost to the recovery path. The trade-off is: less I/O and network bandwidth for WAL (archiving, streaming replication) at the cost of slightly higher CPU on the primary (compression during write) and replica (decompression during replay). `wal_compression` does NOT compress the WAL record headers or non-FPI payload — only the page images. For storage-bound or network-bound replication pipelines, enabling `wal_compression = lz4` (or `zstd` for better ratios) is almost always a net win. Check WAL generation rate via `pg_stat_wal.wal_bytes` before and after enabling to quantify the reduction.

Follow-up Questions:

- How does `wal_compression` interact with `archive_command` — is the archived WAL compressed?
- What is the difference between `wal_compression` and WAL-level compression in pgBackRest?

What Interviewer Looks For: Understanding that `wal_compression` only applies to FPIs and the CPU/I/O trade-off. Knowing PG15 added LZ4/zstd options goes beyond what most candidates know.

Common Mistakes: Believing `wal_compression` compresses all WAL content, not just FPIs. Forgetting that compression adds CPU overhead on recovery, which matters for RTO on large databases.

Q33: Describe the phases of a VACUUM operation on a heap table.

Difficulty: Hard **Category:** VACUUM

Answer: A standard `VACUUM` (non-FULL) proceeds through four main phases. Phase 1 (heap scan): PostgreSQL scans each heap page, identifying dead tuples (those no longer visible to any active snapshot, determined by comparing `t_xmax` against the oldest active transaction's XID from `GetOldestXmin()`). Dead tuple TIDs are accumulated in a memory buffer (`maintenance_work_mem`). Phase 2 (index vacuuming): for each index on the table, `VACUUM` scans the index looking for entries pointing to the dead TIDs collected in phase 1 and removes them. This is the most I/O-intensive phase for large indexes. Phase 3 (heap cleanup): `VACUUM` revisits the heap pages containing dead tuples and marks them `LP_UNUSED`, updates the FSM with newly freed space, and updates the visibility map bits for fully-visible pages. Phase 4 (freeze and statistics): `VACUUM` updates `pg_class.relpages / reltuples`, updates `pg_stat_user_tables`, and performs XID freezing (setting `t_xmin` to `FrozenTransactionId` for old tuples) if their XID age exceeds `vacuum_freeze_min_age`. `pg_class.relfrozensxid` is advanced to reflect the highest XID that has been frozen. `VACUUM ANALYZE` adds an ANALYZE phase after phase 4 to update table statistics.

Follow-up Questions:

- What limits how much of a table `VACUUM` can clean in one pass (hint: `maintenance_work_mem`)?
- How does `VACUUM FULL` differ from regular `VACUUM` in terms of locking and phases?

What Interviewer Looks For: All four phases in order with their purposes. The role of `maintenance_work_mem` as the TID buffer size and the index vacuuming as the most expensive phase are important details.

Common Mistakes: Describing `VACUUM` as a single-pass operation that removes dead tuples and compacts the table like `VACUUM FULL` does. Regular `VACUUM` does NOT compact pages — it only marks space as reusable via the FSM.

Q34: How does the visibility map (VM) accelerate VACUUM and index-only scans?

Difficulty: Hard **Category:** VACUUM

Answer: The visibility map (VM) fork stores two bits per heap page: an "all-visible" bit and an "all-frozen" bit (added in PostgreSQL 9.6). The all-visible bit is set by `VACUUM` when it determines that every tuple on a page is visible to all current and future transactions (i.e., all live tuples have been committed and no dead tuples exist). For `VACUUM`, a page with the all-visible bit set can be skipped entirely in the heap scan phase (only a lightweight check is needed), dramatically reducing `VACUUM` I/O on tables with little churn. For index-only scans, the executor checks the VM bit for each heap page referenced by an index entry: if the bit is set, the heap page does not need to be fetched to verify tuple visibility — the value from the index entry is returned directly, avoiding heap I/O. The all-frozen bit indicates that all tuples on the page have been frozen (`t_xmin` =

FrozenTransactionId), allowing VACUUM FREEZE to skip such pages entirely even when scanning for XID wraparound prevention. Both bits are cleared (set to 0) when any DML modifies the page. `pg_visibility` extension provides per-page visibility map inspection via `pg_visibility(oid)` .

Follow-up Questions:

- What clears the all-visible bit on a page and when is it re-set?
- How does the all-frozen bit interact with `vacuum_freeze_min_age` ?

What Interviewer Looks For: Both bits (all-visible and all-frozen) and their distinct roles. The index-only scan optimization using the VM is often overlooked but is one of the most impactful VM uses.

Common Mistakes: Describing the VM as only having one bit per page (the pre-9.6 all-visible bit) and missing the all-frozen bit. Forgetting that the VM is invalidated by DML, requiring VACUUM to re-scan the page.

Q35: Explain autovacuum tuning. What parameters control when autovacuum triggers?

Difficulty: Hard **Category:** VACUUM

Answer: Autovacuum is triggered per-table when the estimated dead tuple count exceeds:

`autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples` . The defaults are

`autovacuum_vacuum_threshold = 50` (minimum dead tuples before any action) and

`autovacuum_vacuum_scale_factor = 0.2` (20% of live tuples). For a 10-million-row table, autovacuum

triggers after 2,000,050 dead tuples — which may be far too late for a high-churn table. Similarly, ANALYZE

triggers when inserts+updates+deletes exceed `autovacuum_analyze_threshold +`

`autovacuum_analyze_scale_factor * reltuples` (defaults: `50 + 0.1 * reltuples`). For large tables (>100M

rows), scale factors are usually too aggressive; set per-table storage parameters via `ALTER TABLE t SET`

(`autovacuum_vacuum_scale_factor = 0.01`, `autovacuum_vacuum_threshold = 1000`) . Parallelism is

controlled by `autovacuum_max_workers` (default 3) and cost-based throttling via

`autovacuum_vacuum_cost_delay` (default 2ms) and `autovacuum_vacuum_cost_limit` (default 200).

Each page read costs 1, each dirty page write costs 20 (`autovacuum_vacuum_cost_page_dirty`). Cost-based throttling prevents autovacuum from overwhelming I/O on production systems but may prevent it from keeping up on very busy tables.

Follow-up Questions:

- How do you set per-table autovacuum parameters without affecting the whole cluster?
- What is the `autovacuum_freeze_max_age` and how does it force emergency vacuums?

What Interviewer Looks For: The trigger formula with both threshold and scale_factor. Understanding that the 20% default is catastrophic for large tables and that per-table overrides via storage parameters is the solution.

Common Mistakes: Quoting only the scale factor without the threshold, missing that both must be exceeded. Forgetting the cost-based throttling parameters, which are the primary reason autovacuum may "fall behind" on busy tables.

Q36: What is VACUUM FREEZE and when does it run?

Difficulty: Hard **Category:** VACUUM

Answer: `VACUUM FREEZE` (or autovacuum's aggressive `VACUUM` triggered by approaching wraparound) replaces old transaction IDs in tuple headers with the special `FrozenTransactionId` (XID 2), which is treated as "older than all normal XIDs." This prevents XID wraparound by ensuring no live tuple retains a real XID that could wrap around. Regular autovacuum freezes tuples whose `t_xmin` age exceeds `vacuum_freeze_min_age` (default 50,000,000 XIDs) during its normal heap scan. When a table's `pg_class.relfrozensxid` age exceeds `autovacuum_freeze_max_age` (default 200,000,000 XIDs), autovacuum runs an aggressive (full-table) `VACUUM` on that table even if dead tuple counts do not otherwise warrant it — this is the "emergency freeze." PostgreSQL also enforces a hard limit: when `datfrozensxid` (the minimum `relfrozensxid` across all tables in a database) is within `vacuum_failsafe_age` (default 1,600,000,000 XIDs) of wraparound, it enters failsafe mode and disables cost-based throttling on autovacuum. The `age(relfrozensxid)` function returns the current XID age of a table, and `age(datfrozensxid)` from `pg_database` shows the database-wide oldest unfrozen XID.

Follow-up Questions:

- How does `vacuum_freeze_table_age` differ from `vacuum_freeze_min_age` ?
- What is the difference between a multixact freeze and a regular XID freeze?

What Interviewer Looks For: The `FrozenTransactionId` value (XID 2) and the two thresholds: `vacuum_freeze_min_age` (per-tuple age) and `autovacuum_freeze_max_age` (per-table trigger). The failsafe mode in newer versions is a bonus.

Common Mistakes: Confusing the threshold for freezing individual tuples (`vacuum_freeze_min_age`) with the threshold for forcing a full table scan (`autovacuum_freeze_max_age`). Also forgetting that freezing advances `relfrozensxid` , which is what matters for preventing wraparound.

Q37: How does VACUUM interact with long-running transactions?

Difficulty: Hard **Category:** VACUUM

Answer: `VACUUM` cannot remove dead tuples that are still potentially visible to any active snapshot. The oldest active transaction's XID (`GetOldestXmin()`) forms a "horizon" — only tuples with `t_xmax` < horizon (where the deleter has committed) and whose XID is older than all active snapshots can be safely removed. A long-running transaction with `xmin = 100` prevents `VACUUM` from removing any dead tuples created by transactions with `XID > 100`, regardless of how old they are. This causes dead tuple accumulation and table bloat that grows proportionally to the transaction's age. In PostgreSQL 14+, `vacuum_defer_cleanup_age` and `old_snapshot_threshold` provide some control, but the fundamental constraint remains. `pg_stat_activity` shows the `xmin` of each backend (for idle-in-transaction connections, this is the snapshot `xmin`). `pg_stat_replication` shows replica `sent_lsn` and `replay_lsn` — replicas also hold a "replication slot `xmin`" that can block `VACUUM`, especially with logical replication slots (`pg_replication_slots.xmin`). The most dangerous scenario: an abandoned replication slot or a forgotten long-running transaction prevents `VACUUM` from reclaiming any dead tuples, causing unbounded bloat.

Follow-up Questions:

- How do replication slots block `VACUUM` even when no active transaction is running?
- What is `idle_in_transaction_session_timeout` and how does it protect against this problem?

What Interviewer Looks For: The `OldestXmin` horizon concept and its direct connection to dead tuple removal. Mentioning replication slots as another source of `VACUUM` blocking (independent of active transactions) separates expert candidates.

Common Mistakes: Thinking VACUUM is only blocked by active queries, not by idle-in-transaction sessions. Forgetting that logical replication slots can block VACUUM even with zero active user connections.

Q38: What does VACUUM FULL do and when should you use it?

Difficulty: Medium **Category:** VACUUM

Answer: `VACUUM FULL` rewrites the entire table from scratch, compacting all data into a minimal set of pages by excluding all dead tuples and fragmented space. Unlike regular `VACUUM`, which only marks dead space as reusable (not reclaimed from the OS), `VACUUM FULL` returns disk space to the OS by creating a new heap file, copying all live tuples, and replacing the original file. The operation requires an `ACCESS EXCLUSIVE` lock for the entire duration, blocking all reads and writes on the table. It also rebuilds all indexes from scratch. `VACUUM FULL` is appropriate when: a table has undergone massive bulk deletions and the free space will not be reused (reducing disk usage is the goal), or when table bloat has severely degraded sequential scan performance. Alternatives: `pg_repack` (online table rewriting without an exclusive lock) or `CLUSTER` (rewrites in index order, also requires exclusive lock). The new relation file gets a new `relfilenode`, so any open cursors pointing to the old file become invalid. `VACUUM FULL` should be avoided on large tables during business hours due to its lock requirement — `pg_repack` is almost always preferable in production.

Follow-up Questions:

- How does `pg_repack` avoid the `ACCESS EXCLUSIVE` lock that `VACUUM FULL` requires?
- After `VACUUM FULL`, do you need to run `ANALYZE` as well?

What Interviewer Looks For: The key distinction: regular `VACUUM` marks space reusable (FSM), `VACUUM FULL` reclaims disk. Knowing that `VACUUM FULL` holds `ACCESS EXCLUSIVE` lock and blocks all DML is essential for production safety.

Common Mistakes: Recommending `VACUUM FULL` for routine maintenance rather than for specific bloat-recovery situations. Forgetting that `VACUUM FULL` rebuilds indexes, which itself is I/O and CPU intensive.

Q39: How does the `pg_stat_user_tables` view help you monitor VACUUM health?

Difficulty: Medium **Category:** VACUUM

Answer: `pg_stat_user_tables` provides per-table statistics essential for monitoring `VACUUM` effectiveness. Key columns: `n_dead_tup` (estimated dead tuples, updated during `VACUUM` and `ANALYZE`), `n_live_tup` (estimated live tuples), `last_vacuum` (timestamp of last manual `VACUUM`), `last_autovacuum` (timestamp of last autovacuum run), `last_analyze` / `last_autoanalyze` (statistics currency), `vacuum_count` / `autovacuum_count` (cumulative counts), `n_mod_since_analyze` (rows modified since last `ANALYZE`), and `n_ins_since_vacuum` (insertions since last `VACUUM`, PG13+). Monitoring patterns: if `now() - last_autovacuum > 1 hour` for a high-churn table, autovacuum is falling behind — lower `autovacuum_vacuum_scale_factor`. If `n_dead_tup / (n_live_tup + n_dead_tup) > 0.2`, the table has >20% dead tuple bloat. If `last_analyze` is stale (days old on a changing table), the planner is working with outdated statistics. The `age(relfrozenxid)` from `pg_class` combined with `pg_stat_user_tables` gives the full picture: both the operational health (dead tuples, vacuum frequency) and the XID age (wraparound risk). Alerting on `last_autovacuum IS NULL AND n_dead_tup > threshold` catches tables that autovacuum has never touched.

Follow-up Questions:

- How does `pg_stat_user_tables` data persist across server restarts?
- What is the difference between `n_dead_tup` in `pg_stat_user_tables` and the output of `pgstattuple` ?

What Interviewer Looks For: Knowing the key columns (`n_dead_tup`, `last_autovacuum`, `n_mod_since_analyze`) and deriving actionable alerts from them. Combining `pg_stat_user_tables` with `pg_class.relfrozexid` for the complete picture is advanced.

Common Mistakes: Treating `n_dead_tup` as exact rather than estimated (it is updated only during VACUUM/ANALYZE, not in real time). Ignoring `n_mod_since_analyze` as a signal for stale planner statistics.

Q40: What is autovacuum's cost-based throttling and why does it matter?

Difficulty: Hard **Category:** VACUUM

Answer: Autovacuum's cost-based throttling prevents it from consuming excessive I/O resources at the expense of potentially falling behind on large tables. The mechanism assigns a cost to each I/O operation: reading a page from disk costs `vacuum_cost_page_miss = 10`, reading from the buffer pool costs `vacuum_cost_page_hit = 1`, and writing a dirty page costs `vacuum_cost_page_dirty = 20`. Autovacuum accumulates a running cost total and pauses for `autovacuum_vacuum_cost_delay = 2ms` whenever the total exceeds `autovacuum_vacuum_cost_limit = 200`. This yields a maximum effective I/O rate of about 200 cost units per 2ms cycle. At the default settings, a sequential scan of a table not in buffer cache (all page misses at cost 10) allows processing $200/10 = 20$ pages per 2ms cycle, or about 10,000 pages/second — roughly 80 MB/s of heap scan. For a 1 TB table (131,072 8 kB pages), that is $131,072 / 10,000$ seconds ≈ 13 seconds per full scan, which sounds fast but accumulates across all tables in the cluster. Setting `autovacuum_vacuum_cost_delay = 0` disables throttling entirely for aggressive maintenance, though this risks I/O saturation. Per-table overrides via `ALTER TABLE t SET (autovacuum_vacuum_cost_delay = 0)` are the preferred approach for critical tables.

Follow-up Questions:

- How does `vacuum_cost_page_hit` vs `vacuum_cost_page_miss` reflect the actual I/O impact?
- What does PostgreSQL 13's `vacuumdb --parallel` flag do for cost accounting?

What Interviewer Looks For: The cost formula (hit=1, miss=10, dirty=20) and the pause-when-exceeded mechanism. Calculating the effective throughput from the default parameters demonstrates quantitative understanding.

Common Mistakes: Stating that cost-based throttling is disabled by default — the defaults impose significant throttling. Missing that per-table settings allow surgical override for high-priority tables without globally disabling throttling.

Q41: What is a PostgreSQL checkpoint and what does it accomplish?

Difficulty: Medium **Category:** Checkpoints

Answer: A checkpoint is a point in the WAL stream where PostgreSQL guarantees that all dirty shared buffer pages modified up to that point have been written (flushed) to their data files on disk. Its primary purpose is crash recovery efficiency: after a crash, recovery only needs to replay WAL from the most recent checkpoint's

redo LSN rather than from the beginning of WAL. During a checkpoint, the checkpoint process (background daemon since PostgreSQL 9.2) iterates through the shared buffer pool and writes all dirty pages to disk via `write()` and then issues `fsync()` calls to ensure durability. The checkpoint record is then written to WAL, containing the redo LSN and the current state of `pg_control`. `pg_control` is also updated with the checkpoint location. Checkpoints are triggered by: elapsed time (`checkpoint_timeout`, default 5 minutes), WAL volume (`max_wal_size`, default 1 GB — checkpoint when `pg_wal` would exceed this), or explicit `CHECKPOINT` command. A checkpoint consists of two phases: the "spread" phase where dirty pages are written over `checkpoint_completion_target * checkpoint_timeout` to spread I/O, followed by the sync phase which fsyncs all written files.

Follow-up Questions:

- What is the redo LSN vs. the checkpoint LSN and why are they different?
- How does a "restartpoint" differ from a checkpoint on a standby server?

What Interviewer Looks For: The redo LSN concept, the two-phase (write then fsync) structure, and knowing both time-based and WAL-size-based triggers. Understanding `checkpoint_completion_target` as an I/O spreading mechanism is key.

Common Mistakes: Saying a checkpoint writes only WAL, rather than dirty data pages. Confusing the checkpoint's role in crash recovery (bounding recovery time) with its role in PITR (establishing a recovery base).

Q42: How does `checkpoint_completion_target` reduce I/O spikes?

Difficulty: Medium **Category:** Checkpoints

Answer: `checkpoint_completion_target` (default 0.9, range 0.0–1.0) controls what fraction of the inter-checkpoint interval the checkpoint spreads its dirty-page writes over. With `checkpoint_timeout = 5min` and `checkpoint_completion_target = 0.9`, the checkpoint targets completing all dirty-page writes within $0.9 * 5\text{min} = 4.5$ minutes, leaving 0.5 minutes before the next checkpoint fires. This spreading prevents the I/O spike that would occur if all dirty pages were written in a burst at checkpoint time. The checkpoint estimates how many dirty buffers need to be written and divides them evenly across the target interval, adjusting its write rate dynamically via `CheckpointWriteDelay()` which sleeps between page writes. Before PostgreSQL 9.2 (when the dedicated checkpoint process was introduced), checkpoints were performed by the bgwriter, causing more noticeable I/O pauses. Setting `checkpoint_completion_target = 0.9` is standard; higher values risk the checkpoint not completing before the next one fires; lower values increase I/O burstiness. The `pg_stat_bgwriter` (older) or `pg_stat_checkpoint` (PG16+) views show `checkpoints_timed`, `checkpoints_req` (forced by WAL size), and `buffers_checkpoint` to diagnose checkpoint behavior.

Follow-up Questions:

- What does a high `checkpoints_req` count in `pg_stat_bgwriter` indicate?
- How does `max_wal_size` interact with checkpoint frequency?

What Interviewer Looks For: The formula: $\text{target_completion_time} = \text{completion_target} \times \text{interval}$. Understanding that `checkpoints_req` being high means `max_wal_size` is too small and checkpoints are WAL-bound rather than time-bound.

Common Mistakes: Believing that setting `checkpoint_completion_target = 1.0` is optimal — it actually risks the checkpoint running into the next interval. Conflating the bgwriter's continuous background writes with the checkpoint's periodic bulk writes.

Q43: What does the background writer (bgwriter) do and how is it different from the checkpointer?

Difficulty: Medium **Category:** Checkpoints

Answer: The background writer (bgwriter) continuously scans the shared buffer pool's LRU clock and proactively writes clean (unflushed) dirty buffers to disk so that they are available for reuse without backend processes needing to write them. This reduces latency for user-facing queries that would otherwise stall waiting for buffer eviction. The bgwriter does NOT fsync — it only writes pages to the OS page cache. Key parameters: `bgwriter_lru_maxpages` (default 100, max dirty pages written per round), `bgwriter_lru_multiplier` (default 2.0, aggressiveness multiplier for predicting how many clean buffers to prepare), and `bgwriter_delay` (default 200ms, sleep between rounds). The checkpointer, by contrast, is a separate process responsible for periodic checkpoint writes with fsync — it writes ALL dirty pages at checkpoint time and ensures durability. The bgwriter's goal is latency reduction for normal operations; the checkpointer's goal is crash recovery safety and WAL retention management. Both processes write to the same shared buffer pool but with different objectives and fsync behavior.

`pg_stat_bgwriter.buffer_clean` (bgwriter writes) vs `buffer_checkpoint` (checkpointer writes) vs `buffer_backend` (backend direct writes) diagnose which path dominates.

Follow-up Questions:

- What does a high `buffer_backend` count indicate and why is it a problem?
- How does the LRU clock (clock-sweep) algorithm work for buffer eviction?

What Interviewer Looks For: The bgwriter writes without fsync; the checkpointer fsyncs. The role of bgwriter in reducing backend stalls (`buffer_backend`) vs checkpointer's durability role is the core distinction.

Common Mistakes: Believing the bgwriter performs fsyncs. The bgwriter only calls `write()`, not `fsync()` — only the checkpointer and WAL writer perform fsyncs in their respective domains.

Q44: What triggers a forced (non-scheduled) checkpoint and what are its consequences?

Difficulty: Hard **Category:** Checkpoints

Answer: Forced checkpoints (shown as `checkpoints_req` in `pg_stat_bgwriter` / `pg_stat_checkpoint`) are triggered when WAL generation outpaces the scheduled checkpoint interval. Specifically, when the size of WAL generated since the last checkpoint exceeds `max_wal_size` (default 1 GB), PostgreSQL fires an immediate checkpoint regardless of `checkpoint_timeout`. Other forced checkpoint triggers include: explicit `CHECKPOINT` command (DBA-initiated), `pg_start_backup()` (initiates a backup checkpoint), server shutdown sequence, and recovery completion. Forced checkpoints are problematic because they do not benefit from the spreading mechanism of `checkpoint_completion_target` — they must complete quickly, causing a burst of dirty-page writes and fsyncs that can temporarily saturate I/O and increase query latency. To diagnose: monitor `checkpoints_req / (checkpoints_req + checkpoints_timed)` — if this ratio exceeds 10–20%, `max_wal_size` is too small for the workload. Remediation: increase `max_wal_size`, reduce write amplification, or increase `checkpoint_timeout`. On systems with `checkpoint_timeout = 5min` and `max_wal_size = 1GB` under heavy write load, both triggers may fire frequently.

Follow-up Questions:

- How do you calculate the right `max_wal_size` for a given write workload?
- What role does `min_wal_size` play and how does it interact with `max_wal_size` ?

What Interviewer Looks For: All forced-checkpoint triggers (not just the WAL-size trigger) and the lack of spreading as the key consequence. Knowing the `checkpoints_req` metric for diagnosis is a practical production skill.

Common Mistakes: Listing only the WAL-size trigger as a forced checkpoint cause, forgetting backups, explicit commands, and shutdown. Underestimating the performance impact of forced checkpoints on query latency.

Q45: Describe the PostgreSQL query pipeline from SQL text to result rows.

Difficulty: Medium **Category:** Query Pipeline

Answer: The PostgreSQL query pipeline consists of five stages: (1) Parsing: the raw SQL string is tokenized by the lexer and parsed by the Bison grammar into a parse tree (`raw_parser()`), which is an unanalyzed abstract syntax tree with identifiers as strings. (2) Analysis/Semantic analysis: `parse_analyze()` resolves table and column names against the system catalogs (`pg_class` , `pg_attribute` , etc.), type-checks expressions, resolves function overloads, and produces a `Query` struct (query tree). OIDs replace string identifiers at this stage. (3) Rewriting: the rule system (`QueryRewrite()`) applies any `CREATE RULE` definitions, expanding views into their underlying queries, and handles `ON SELECT/INSERT/UPDATE/DELETE` rules. (4) Planning/Optimization: `planner()` takes the rewritten `Query` tree and produces a `PlannedStmt` containing a tree of `Plan` nodes (SeqScan, IndexScan, Hash, NestLoop, etc.) with estimated row counts, costs, and join strategies. (5) Execution: `PortalRun()` drives the `ExecProcNode()` recursive descent through the plan tree, fetching rows from storage and applying operators. Each stage's output is a well-defined data structure, making the pipeline modular and extensible.

Follow-up Questions:

- At which stage does PostgreSQL detect a syntax error versus a semantic error?
- How does a prepared statement (`PREPARE` / `EXECUTE`) interact with the pipeline stages?

What Interviewer Looks For: All five stages in order (parse → analyze → rewrite → plan → execute) with the correct data structures (raw parse tree → `Query` → `PlannedStmt`). Distinguishing syntax errors (stage 1) from semantic errors (stage 2) shows depth.

Common Mistakes: Omitting the rewrite stage entirely or conflating it with planning. Forgetting that parse trees use string identifiers while analyzed query trees use catalog OIDs.

Q46: How does the PostgreSQL planner estimate row counts for a sequential scan with a WHERE clause?

Difficulty: Hard **Category:** Query Pipeline

Answer: Row count estimation for a sequential scan with a WHERE clause involves the statistics stored in `pg_statistic` (exposed via `pg_stats`). For an equality condition `col = const` , the planner calls `clause_selectivity()` , which looks up the column's statistics: `n_distinct` (number of distinct values, negative = fraction of total rows), `most_common_vals` (MCVs, an array of frequent values), `most_common_freqs` (corresponding frequencies), and `histogram_bounds` (bucket boundaries for non-

MCV values). If `const` appears in `most_common_vals`, its frequency is used directly. If not, the remaining probability mass ($1 - \text{sum}(\text{MCVs freqs})$) is divided equally across the non-MCV distinct values to estimate the selectivity. For range conditions, the planner uses the histogram: it finds which bucket `const` falls in and interpolates. For multi-column predicates, selectivities are multiplied (independence assumption) unless extended statistics exist (`CREATE STATISTICS ... (dependencies)`). The estimated selectivity is multiplied by `pg_class.rel tuples` to get the estimated row count. `EXPLAIN (ANALYZE)` shows `rows=N` (estimated) vs. `actual rows=M` to expose estimation errors.

Follow-up Questions:

- What is the `default_statistics_target` and how does increasing it improve estimates?
- How do extended statistics with `CREATE STATISTICS` improve multi-column estimates?

What Interviewer Looks For: The MCV lookup first, then histogram interpolation, then independence multiplication for multi-column predicates. Knowing `pg_statistic` column names (`most_common_vals`, `histogram_bounds`) shows source-level familiarity.

Common Mistakes: Describing statistics as just a row count without mentioning MCVs and histograms. Forgetting the independence assumption for multi-column predicates and not knowing that extended statistics address this.

Q47: Explain the PostgreSQL cost model. What are the key cost parameters and what do they represent?

Difficulty: Hard **Category:** Query Pipeline

Answer: PostgreSQL's cost model assigns abstract cost units to each plan node operation, representing relative resource consumption. Key cost parameters: `seq_page_cost = 1.0` (baseline cost of reading one page sequentially from disk, the reference unit), `random_page_cost = 4.0` (default, cost of random page access — reflects seek time for HDDs; should be ~1.1 for SSDs), `cpu_tuple_cost = 0.01` (cost of processing one tuple in a node), `cpu_index_tuple_cost = 0.005` (cost of processing one index entry), `cpu_operator_cost = 0.0025` (cost of evaluating one operator/function). A sequential scan's cost is `seq_page_cost * relpages + cpu_tuple_cost * reltuples`. An index scan's cost includes `random_page_cost * pages_fetched` (heap fetches) plus the index traversal cost. The planner minimizes total cost across join orderings (using dynamic programming for `<= join_collapse_limit` tables, or GEQO for more). `effective_cache_size` (default 4 GB) does NOT reserve memory but tells the planner how much OS page cache is available, influencing whether random I/O is penalized (pages likely in cache → lower effective random cost). Misconfigured `random_page_cost` (keeping 4.0 on fast SSDs) is one of the most common causes of suboptimal plan choices.

Follow-up Questions:

- How does `parallel_tuple_cost` and `parallel_setup_cost` factor into parallel query plans?
- What is the difference between startup cost and total cost in `EXPLAIN` output?

What Interviewer Looks For: The five core cost parameters with correct defaults and semantic meanings. Knowing that `random_page_cost = 4.0` is wrong for SSDs and causes index scans to be under-selected is a critical practical insight.

Common Mistakes: Saying `effective_cache_size` allocates memory rather than just informing the cost model. Confusing `seq_page_cost` and `random_page_cost` units (both are abstract, not milliseconds or bytes).

Q48: What are extended statistics and what types does PostgreSQL support?

Difficulty: Hard **Category:** Query Pipeline

Answer: Extended statistics (introduced in PostgreSQL 10, expanded in later versions) address the independence assumption that the planner makes when estimating multi-column predicate selectivity. Without extended statistics, `WHERE a = 1 AND b = 2` is estimated as `sel(a=1) * sel(b=2)`, which is incorrect when `a` and `b` are correlated. `CREATE STATISTICS name ON col1, col2 FROM table` collects multi-column statistics. Three types are supported: (1) `ndistinct` — estimates the number of distinct value combinations for multiple columns, improving `GROUP BY` and `DISTINCT` estimates; (2) `dependencies` — captures functional dependencies (e.g., `city` implies `country`), improving `WHERE` clause estimates for correlated columns; (3) `mcv` (Most Common Values, added in PG12) — a list of the most common multi-column value combinations with their joint frequencies and null fractions, providing the most accurate selectivity estimates for correlated columns. Types can be specified: `CREATE STATISTICS s1 (dependencies, ndistinct) ON a, b FROM t`. After creation, `ANALYZE` populates the statistics, stored in `pg_statistic_ext` and `pg_statistic_ext_data`. `EXPLAIN` with `format=json` shows `Rows Removed by Filter` to diagnose plan quality.

Follow-up Questions:

- How many extended statistics objects can you create on a single table?
- What is the performance overhead of collecting extended statistics during `ANALYZE`?

What Interviewer Looks For: All three types (`ndistinct`, `dependencies`, `mcv`) with correct version introductions. Understanding that the planner uses `pg_statistic_ext_data` to override the independence assumption is key.

Common Mistakes: Knowing only `dependencies` as the single type of extended statistics, missing `ndistinct` and `mcv`. Forgetting that extended statistics require `ANALYZE` to be run after creation before they are used by the planner.

Q49: How does PostgreSQL's parallel query execution work at the process level?

Difficulty: Hard **Category:** Query Pipeline

Answer: PostgreSQL parallel query uses a leader/worker process model based on dynamic background worker processes. When the planner decides a parallel plan is beneficial, it inserts `Gather` or `Gather Merge` nodes above parallel-capable scan/join nodes. At execution time, the leader process executing `ExecProcNode` on a `Gather` node launches up to `max_parallel_workers_per_gather` (default 2) parallel worker processes via `LaunchParallelWorkers()`. Workers are drawn from a pool of pre-forked processes (up to `max_parallel_workers`, default 8, which is a subset of `max_worker_processes`, default 8). Workers and the leader share a DSM (Dynamic Shared Memory) segment containing: a `ParallelContext` struct, a tuple queue (DSA-based parallel queue) per worker for sending tuples back to the leader, and relation-level scan state (e.g., the block range counter for parallel sequential scans). Each worker calls the same plan subtree independently, with a parallel-aware heap scan dividing blocks via an atomic counter in shared memory. The leader also participates as a worker. `Gather Merge` sorts the worker outputs maintaining sort order. Parallel workers have their own transaction context but share the parent's snapshot via shared memory copy.

Follow-up Questions:

- Why can't PostgreSQL parallelize certain operations like `INSERT INTO ... SELECT` ?
- How does `parallel_leader_participation` affect Gather node behavior?

What Interviewer Looks For: The DSM shared segment, the atomic block counter for scan division, and the Gather/Gather Merge distinction. Knowing the three-level limits (`max_parallel_workers_per_gather`, `max_parallel_workers`, `max_worker_processes`) is operational depth.

Common Mistakes: Describing parallel query as multi-threaded rather than multi-process. Forgetting that the leader also participates as a parallel worker (not just a coordinator), which is controlled by `parallel_leader_participation` .

Q50: How do system catalog tables (`pg_class`, `pg_attribute`, `pg_index`) relate to each other and what information do they store?

Difficulty: Hard **Category:** System Catalogs

Answer: The PostgreSQL system catalog is a set of regular heap tables in the `pg_catalog` schema that describe all database objects. `pg_class` is the central catalog with one row per relation (table, index, view, sequence, materialized view, composite type, TOAST table, etc.), storing: `oid` (object identifier), `relname` (name), `relnamespace` (OID of `pg_namespace` schema), `reltype` (OID of row type in `pg_type`), `relfilenode` (physical file name, 0 for views), `reltablespace` , `relpages` (VACUUM-estimated page count), `reltuples` (VACUUM-estimated row count), `relhasindex` , `relkind` (r=table, i=index, v=view, m=materialized view, S=sequence, t=TOAST, p=partitioned), `relfrozenxid` , and storage parameters. `pg_attribute` has one row per column, keyed by (`attrelid` → `pg_class.oid` , `attnum`) storing column name, type OID (`atttypid` → `pg_type`), length, position, default OID, statistics target, and the TOAST storage strategy. `pg_index` has one row per index, storing `indexrelid` (the index's `pg_class` OID), `indrelid` (the heap table's OID), `indkey` (int2vector of attribute numbers), `indisprimary` , `indisunique` , `indpred` (partial index predicate), and `indexprs` (expression index trees). These three catalogs join as: `pg_class` (index) ← `pg_index.indexrelid` / `indrelid` → `pg_class` (table) → `pg_attribute` (columns).

Follow-up Questions:

- How do you find all partial indexes on a table using the system catalogs?
- What does `pg_class.relfilenode = 0` indicate for a relation?

What Interviewer Looks For: Naming the specific columns (`relfilenode`, `relfrozenxid`, `indkey`, `indpred`) and knowing how the three catalogs join. Understanding `relkind` values (r/i/v/m/S/t/p) differentiates deep catalog knowledge.

Common Mistakes: Describing `pg_class` as only storing table metadata, forgetting it covers all relation types including indexes, views, and TOAST tables. Confusing `pg_attribute.attnum` with an array index — `attnum` is the column position starting at 1, with negative numbers for system columns.
